

UMEÅ UNIVERSITY
Department of Physics
Modeling and Simulation, 7.5hp

October 1, 2026

Molecular Dynamics, Stochastic simulations, and Monte Carlo

Peter Olsson

Miscellaneous comments

- The following instructions are intended to be used for a Ubuntu/Linux system. If you are in the computer lab you might need to restart the computer. It will then automatically boot into Ubuntu.
- Open up the lab instructions from the browser and open up a terminal which is what you run the program from. You will now be able to copy things directly from the lab instructions to the terminal. Note that copying text is done by just pressing and moving the left mouse button, nothing more. To paste the content to the terminal: Move the mouse over the terminal, activate the terminal by left-clicking and paste by pressing the middle mouse button.
- Send the report through Canvas.
- Provide a path to the relevant code on the computers, or make it available by some other means.
- There are functions in `ran.c` for getting different kinds of random numbers.

1 Introduction

A gas of particles is a very important system in physics. If the density is low, one can neglect collisions between particles. When that is a good approximation we say that the gas behaves like an ideal gas and is then well described by the ideal gas law,

$$pV = Nk_B T,$$

which is a relation between pressure p , volume V , number of particles N , temperature T , and Boltzmann's constant, $k_B \approx 1.38 \times 10^{-23}$ Joule/Kelvin.

1.1 Interaction

The fact that gases liquify at higher densities is due to interactions between the molecules. This interaction is repulsive at short distances and attractive at somewhat larger distances and is commonly modelled with the Lennard-Jones interaction,

$$u_{\text{LJ}}(r) = 4\epsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]. \quad (1)$$

Here the length σ and the energy ϵ are material parameters. We will however use units such that they are both equal to unity. To save time we will cut off the interaction for $r > r_c$ and use

$$u_s(r) = \begin{cases} u_{\text{LJ}}(r) - u_{\text{LJ}}(r_c), & r < r_c, \\ 0, & r > r_c. \end{cases}$$

The cutoff distance is specified in `define.h` as

```
#define CUT 3
```

The force between the particles is given by $F = -du_s/dr$. A positive sign means a *repulsive* force.

$$F(r) = \frac{\epsilon}{r} \left[48 \left(\frac{\sigma}{r} \right)^{12} - 24 \left(\frac{\sigma}{r} \right)^6 \right]. \quad (2)$$

The above expression gives the magnitude of the force; to capture the vector nature we write (in two dimensions)

$$\mathbf{F} = -\nabla u_s = -\frac{du}{d\mathbf{r}} = -\frac{dr}{d\mathbf{r}} \frac{du_s}{dr} = -\left(\frac{dr}{dx}, \frac{dr}{dy} \right) \frac{du_s}{dr} = \left(\frac{x}{r}, \frac{y}{r} \right) F(r). \quad (3)$$

The last equality follows from differentiation of $r^2 = x^2 + y^2$ which gives

$$r \, dr = x \, dx + y \, dy.$$

1.2 Molecular dynamics

The dynamics of a system with N particles is governed by two coupled differential equations,

$$\begin{aligned}\dot{\mathbf{r}}_i &= \mathbf{v}_i, \\ m\dot{\mathbf{v}}_i &= \mathbf{F}_i,\end{aligned}$$

where $\mathbf{F}_i$ is the sum of the forces acting on particle i due to the interaction with the other particles. In the simulations this should be integrated with a small time step, Δt , and with the notation $\mathbf{v}_i^{(n)} = \mathbf{v}_i(n\Delta t)$ the leap-frog method becomes

$$\mathbf{v}_i^{(n+1/2)} = \mathbf{v}_i^{(n-1/2)} + \frac{1}{m}\mathbf{F}_i^{(n)} \Delta t, \quad (4)$$

$$\mathbf{r}_i^{(n+1)} = \mathbf{r}_i^{(n)} + \mathbf{v}_i^{(n+1/2)} \Delta t. \quad (5)$$

We will use simulation units which means that we set $\sigma = 1$, $m = 1$, $\epsilon = 1$ and $k_B = 1$, and let $k_B T \rightarrow T$.

2 A program for Molecular dynamics and Langevin dynamics

We are here going to simulate interacting particles in two dimensions. An important reason for looking at this problem in two dimensions is that it then becomes much easier to visualize the configurations. This is also one of the voluntary exercises.

Your task is now to complete an almost finished program.¹ The first step is to look through the program *carefully* to understand how it is meant

¹Some of you could prefer to instead write everything from scratch and would perhaps also do that successfully. This exercise could anyway be a good one since the modification of an existing code is more common than writing things from scratch, which means that there is a fair chance that you will be asked to modify an existing program in your coming professional activities.

to work. Check both the `Makefile` and all the source files. This is also a good opportunity to learn a few programming tricks. The program begins (as always) in the function `main` which is at the bottom of `sim.c`.

2.1 Getting the code

Open a terminal by pressing Ctrl-Alt-t, make a new directory

```
$ mkdir LabStoch
```

and go to that directory by writing

```
$ cd LabStoch
```

Quite a few files are needed to complete this lab and they are stored in a compressed archive at www.tp.umu.se/modsim/files/LabStoch-2025.tgz. The command line can be used to download this file (though it can of course also be done with a web browser):

```
$ wget www.tp.umu.se/modsim/files/LabStoch-2025.tgz
```

To get all the files together with some directories, execute²

```
$ tar xzf LabStoch-2025.tgz
```

which is the command to extract the content from the gzipped (compressed) tar file. To see what files and directories are present type

```
$ ls -l
```

which gives a listing of the directory. In the following you should edit files in `src` and compile in `lang`.

²In computing, `tar` (derived from tape archive) is both a file format (in the form of a type of archive bitstream) and the name of the program used to handle such files. “.tgz” is the same as “.tar.gz”—a gzipped (compressed) tar file.

2.2 Completing the code

Note that appendix A gives a short introduction to the program.

Follow the steps below to convince yourself that your program is correct. In the source code the incomplete sections are marked with “**Fix this (1)**”, where the numbers, 1–5, correspond to the numbers below. This part is only to become familiar with the workings of the program and should not be included in the report.

For editing the source code with the standard text editor, `gedit`, type

```
$ gedit src/sim.c &
```

This will open a new window that shows `sim.c`. The `&` character is needed to be able to continue using the terminal for other commands. The shell (which is the program running in the terminal and accepting input) will otherwise wait for `gedit` to terminate before showing the prompt, “\$”, which shows that it accepts input. Then go to directory `lang`

```
$ cd lang
```

Note that `Makefile` in that directory contains `VPATH = ../src` which instructs the `make` program to look for source files in the source directory of the current directory tree.

It should now be possible to compile the program. When running

```
$ make
```

you see a number of lines for compiling source files into object files followed by a single line that links together these object files in the the executable file `sim`.

```
gcc -g -O3 -I. -c -o sim.o ../src/sim.c
...
gcc sim.o ran.o common.o config.o measure.o dynamics.o -lm -o sim
```

If you type “`$ ls -l`” in the terminal you will see that there is now a file `sim` which is “executable”, which is the meaning of the “x” in “-rwxr-xr-x”. Now run the program by typing (or copying from this document)

```
./sim N=64 rho=0.5 T=1.0 read=0064_r0.500_T1.000_start run
```

The program then starts and first tells that it has managed to read a configuration file with initial values of positions and velocities, and also gives values for both potential and kinetic energies,

```
Read from conf/0064_r0.500_T1.000_start with data for 64 particles
Potential energy = 0
Kinetic energy = 1.00701
...exiting
```

The reason for the zero Potential energy is that the program lacks quite a few parts and your task is now to complete the following steps to get a working code:

1. Complete the functions `measure` (in `measure.c`) and `pair_interaction` (in `common.c`) which are necessary to perform the measurements in the program. Appendix A gives some information about the layout of the array `atoms` used for storing both position and velocity coordinates. Compile the program by typing `make` in the terminal. After running the program again with the same command you will hopefully see the text below:

```
Potential energy = -1.30942
Kinetic energy   = 1.00701
```

Note that these are intensive values, energy per particle.

2. Next, remove the `exit(EXIT_SUCCESS)` in `sim.c` that terminated the program. Study the function `step` (in `dynamics.c`) and complete the functions `force_magnitude` and `one_force` in `common.c` by using Eq. (2) and Eq. (3). We want a code that should be useful for different number of dimensions and use 'D' for the dimensionality. (This is specified in `lang/define.h`: `#define D 2`.)

Also do a change in `vel_from_force` (`dynamics.c`) to implement the leap-frog method.

If not specified otherwise the friction parameter is $\alpha = 0$, which means that the program by default does molecular dynamics. With the same command line as before you should now get the following output for the positions and velocities for particle number 0:

```
x, y, vx, vy = 7.5937  7.58183  -0.539249  -1.35814
```

3. Remove the print statement for `x, y, vx, vy = ...` and the `exit` statement. The program will then instead run `par->ntherm` units of time for equilibrium and the program is now ready for long runs.
4. We want both averages and standard errors from the collected data and that is prepared for in the code. Note that the program makes use of structured variables

```
Measured *val, *vblock, *v1sum, *v2sum;
```

and because of the definition of the `struct` in `sim.h` this means that we have the variables `val->epot`, `val->pressure`, `val->ekin`, and `val->etot`, and similarly for the struct `vblock` that contains the accumulated measurements.

There is no way to directly loop over structured variables, e.g. to accumulate `val->epot` into `vblock->epot`, and to do the same with the other measured quantities, but this is actually achieved by introducing the array `dval` which has the same address as the struct `val`,

```
double *dval;
dval = (double *) val;
```

The construct

```
for (iv = 0; iv < nval; iv++)
    dvblock[iv] += dval[iv];
```

then means that all the values in `val` are accumulated into `vblock`. The same construct has also been used for `v1sum` and `v2sum`.

With `v1sum[]` and `v2sum[]` (see how they are calculated in the code) we may complete the function `standard_error()` to calculate averages and estimated standard errors according to the following formulas:

$$X = \frac{1}{\text{nblock}} \sum_i x_i, \quad X_2 = \frac{1}{\text{nblock}} \sum_i x_i^2, \quad \text{stderr}(X) = \sqrt{\frac{X_2 - X^2}{\text{nblock} - 1}}.$$

This `stderr(X)` is a quantity that decreases when the run is longer, i.e. the number of blocks is bigger.

You will also need to determine the typical size of the fluctuation of the energy which is given by

$$\sigma_E = \sqrt{\langle E^2 \rangle - \langle E \rangle^2}$$

As this is a measure of the size of the fluctuations it shouldn't (and doesn't) decrease with an increasing number of blocks. In order to do that you need to get the average of E^2 *from the individual measurements*. Do that by introducing a variable `val->etot2`. (To check that your values of σ_E are correct you can also determine the same quantity from the energy values in the files in directory `efile`.)

5. The program should now perform (deterministic) Molecular dynamics. To change this to the stochastic Langevin dynamics you need to complete `langevin_forces` (in `common.c`). You should then let the random number ζ in each direction be given by $C_{\text{Lang}} \times \xi_{\pm}$ where $\xi_{\pm}$ is from a uniform distribution between -1 and 1 . This is the kind of random number returned by `dran_sign()`. Note that $\langle \xi_{\pm}^2 \rangle = 1/3$. Adjust C_{Lang} to get the correct magnitude of the noise (see Sec. 3.2 for details) and run the program with $\alpha = 0.1$ to check that the kinetic energy per particle is equal to T , which is $T/2$ per degree of freedom (= dimension). If that is not so, please contemplate the meaning of $\langle \xi_{\pm}^2 \rangle = 1/3$, before asking the lab tutor.

The program should now be ready for the following exercises.

3 Molecular dynamics and Langevin dynamics

3.1 Molecular dynamics and the time step

In molecular dynamics it is important to check that the time step is not too big. To that end one looks at the total energy which should be a constant. Consider both *stability* and *precision*. The files in directory `efile` contain columns with time and total energy per particle, $E = (U + K)/N$.

Exercise 3.1:

1. Do a number of runs, starting from `conf/0064_r0.500_T1.000_start`, with $\alpha = 0$ with 20 blocks (`nblock=20`) in each run and time steps, $\Delta_t = 0.002, 0.004, 0.006, 0.008, 0.010$ and 0.012 . The time step is specified with e.g. `deltat=0.002` on the command line. Plot both $\langle E \rangle$ vs Δ_t and σ_E vs Δ_t .
2. To see clearly the possibility of an *instability* in the simulations, do a run with 50 blocks and $\Delta_t = 0.016$. Plot E vs t , restricting the y -axis to between -0.4 and 1.0 . (There is no point in showing the behavior as $E(t)$ diverges.)

3.2 Langevin dynamics — the effect of a finite α

In Langevin dynamics the total energy fluctuates and it is interesting to examine how these fluctuations depend on α . In this dynamics both damping and noise give changes to the velocity:

$$m\mathbf{v}_i(t + \Delta_t) = m\mathbf{v}_i(t) + (\mathbf{F}_i - \alpha\mathbf{v}_i + \boldsymbol{\zeta}_i)\Delta_t. \quad (6)$$

Here $\boldsymbol{\zeta}_i$ is a vector noise where each component has the magnitude

$$\langle \zeta^2 \rangle = \frac{2\alpha T}{\Delta_t}.$$

Exercise 3.2: The question in focus is now what changes and what remains the same when we change α . Do runs with $\alpha = 0.01, 0.1$, and 1.0 , density $\rho = 0.6$ and temperature $T = 1.0$ and plot energy vs time. Skip the statement

`read=conf/0064_r0.500_T1.000_start`, since that is meant to work with $\rho = 0.5$. It is now OK with shorter runs, say 10 blocks. We take $\Delta_t = 0.01$ which gives a reasonable precision (as shown above), and is also the default value of the time step in the code. Consider both the size and the velocity of the energy fluctuations:

1. Examine the size of the fluctuations, σ_E . Do they change with α ? What would you expect from theory? (For small α it might be necessary to do longer runs to get a good measure of the size of the fluctuations.)
2. For this examination you will then need to change the writing to the energy file (`efile`) such that it happens after each measurement. In the given code it is only every 100th measurement which is written to file. This is the effect of “`if (isamp % 100 == 0)`” before printing to file. (Usually we do, however, not like to print each measured energy value to file, as it produces big files.)

Plot E vs t . Comment on the energy fluctuations—are they rapid or slow?—and relate them to the fact that m/α has the dimension of time.

4 Brownian dynamics

Go to directory `brown` to use for Brownian dynamics and copy the files:

```
$ cd ..
$ cp lang/define.h lang/Makefile brown
$ cd brown
```

You then need to change in the new `define.h`:

- Remove `#define VEL` since Brownian dynamics only operates on the positions.
- Instead include `#define BROWN` which should be used in the code (mostly in the function `step` in `common.c`) for selecting statements to use for Brownian dynamics.

The dynamics is given by

$$\mathbf{r}_i(t + \Delta_t) = \mathbf{r}_i(t) + \mathbf{F}_i \frac{\Delta_t}{\alpha} + \boldsymbol{\eta}_i \Delta_t,$$

where each component of the noise is characterized by

$$\langle \eta^2 \rangle = \frac{2T}{\alpha \Delta_t}.$$

As a quick test that the program is correct, check that the obtained potential energy when using $\alpha = 10.0$ and $\Delta_t = 0.001$ gives about the same results as the Langevin program.

5 Monte Carlo

The last exercise is to make another program that does Monte Carlo. Go to the `mc` directory, copy `define.h` and `Makefile`, and put `#define MC` in `define.h`.

The changes to implement Monte Carlo will mainly be to write a function in `monte_carlo.c` that performs a Monte Carlo sweep. Note that you should also keep track of the acceptance ratio and that there is a parameter b that specifies the maximum distance to move the particles.

A Monte Carlo sweep is done by looping over the particles i and for each i do the following operations:

1. Suggest a new position: $\mathbf{r}'_i = \mathbf{r}_i + b\boldsymbol{\delta}$, where $\boldsymbol{\delta}$ is a random vector.
2. Use ν for the original configuration and μ for the suggested one, and calculate the energy difference $\Delta U = U_\mu - U_\nu$ and

$$\alpha_{\nu \rightarrow \mu} = \min(1, e^{-\Delta U/T}).$$

3. To accept with this probability, generate a random number ξ and accept if $\xi < \alpha_{\nu \rightarrow \mu}$, which means that we let $\mathbf{r}'_i$ be the new $\mathbf{r}_i$.

Note that there is no need to calculate the full U_μ and U_ν (which are order- N^2 calculations) to determine ΔU , but that one can instead use

$$\Delta U = \sum_{j|j \neq i} u(\mathbf{r}'_i - \mathbf{r}_j) - u(\mathbf{r}_i - \mathbf{r}_j), \quad (7)$$

which is a calculation of order N .

5.1 Comparison

We will now demonstrate that Brownian motion gives the same results as Monte Carlo in the limit of small Δ_t/α :

Exercise 5.1: Make a long run with the Monte Carlo program to get good precision for the potential energy with the parameters $N = 64$, $\rho = 0.3$, and $T = 1.0$. Run the Brownian dynamics program with $(\alpha, \Delta_t) = (10.0, 0.001)$, $(10.0, 0.002)$, $(10.0, 0.003)$, and $(10.0, 0.004)$. Plot the potential energy *with error bars* vs Δ_t/α . Put the Monte Carlo data at $\Delta_t/\alpha = 0$. Be sure to run long enough to get reasonably precise data, which in this case means that the statistical errors ought to be < 0.002 . It should be possible to see a clear trend in the data.

6 Voluntary exercises

For (at least some of) these voluntary exercises it could be convenient not to write out the result from the simulations to the terminal, only, but to also write them to files, e.g. in a directory `res`, with file names given by the parameters, just as the final configuration is written to `conf/filename`. When doing this, be sure to write both the parameters of the run and the average values (and perhaps also the standard error) to the files and to make files that are easily read into your plotting program (e.g. `matlab`).

Note also that there are some suggestions in Appendix B for speeding things up by running several simulations at the same time.

6.1 Different phases

At high temperatures (here, around or above $T = 1$) the system is a gas with a uniform density. When the temperature is lowered the attractive interaction starts to become more important and the particles prefer to be close to one another. This will then lead to a denser region (a liquid) coexisting with the region with lower density (a gas). The liquid is however about as disordered as the gas. If the temperature is lowered even further the system will try hard to get the lowest possible energy, which is obtained by forming an ordered crystal. The transition between such phases may be studied with advanced methods, but we will here just look at snapshots from the stored configuration files.

Do Monte Carlo at temperatures $T = 1.0, 0.9, 0.8, \dots 0.1$ on a larger system with $N = 200$ and $\rho = 0.25$. Be sure to start the run at the lower temperature from the configuration of the next higher one. Show representative pictures of (1) gas, (2) gas-liquid coexistence, and (3) solid phase (well, gas-solid coexistence), and try to approximately locate the phase transition temperatures.

6.2 Speeding things up with a neighbor list

Our systems with $N = 64$ particles is exceptionally small; it is more common with simulations with tens or hundreds of thousands particles. For that case the double loop that is present at several places in the code

```
for (i = 0; i < par->n; i++)
```

```
for (j = i + 1; j < par->n; j++)
```

does however become problematic since there are then too many calculations. The way out is to create a list of particle indices, for each particle i , that contains the particles that are reasonably close to that particle together with another array with the number of neighbors. After initializing these lists the above double loop could instead be

```
for (i = 0; i < par->n; i++)
  for (jj = 0; jj < neighbor_count[i]; jj++) {
    j = neighbor_list[nnsizes[i] * i + jj];
```

and if the typical value of `neighbor_count[i]` is around 50 instead of N , this will mean a big speedup for large N .

Since we want the neighbor list to be useful even when the particles have moved somewhat one has to also include some particles with $r_{ij} > r_c$. One therefore introduces a parameter Δr for this additional distance and lets the neighbor list for i include all particles j with $r_{ij} < r_c + \Delta r$.

To check for the need to recalculate the neighbor list one then examines the maximum particle displacement since the creation of the neighbor list, and if it is greater than $0.5\Delta r$ might be possible that there is a particle j which now has a distance $r_{ij} < r_c$ even though it is not in the neighbor list of i , and this certainly means that the neighbor list has to be recalculated. This check has to be done before each dynamics step.

Verify the code To check that this faster code really is correct one should make identical runs with the same starting configuration and the same sequence of random numbers. To get the same random number one specifies the seed to the random number generator (which is otherwise taken from the clock) by specifying e.g. `seed=1` on the command line.

Timing the run To determine the time needed for the run, just put `time` before the command itself. The output then looks like

```
real    0m1.453s
user    0m1.450s
sys     0m0.003s
```

where `real` is the wall clock time, `user` is the actual CPU time used in executing the process whereas `sys` is the amount of CPU time spent in the kernel within the process.

Make a directory `mc-fast` where you put `#define FAST` into `define.h`. Complete the functions `neighbors_list.c` and make fast code alternatives in both `monte_carlo.c` and `measure.c`. Do some (rather short) runs with $N = 1024$ and $N = 4096$ particles and compare the needed simulation times. Also determine the average neighbor list size and the number of sweeps between successive recalculations of the neighbor lists.

Remark The function for creating the neighbor list described above does make use of the double loop and is therefore of order N^2 and therefore quite slow. It is however possible to speed it up by splitting the system up into a number of boxes and make a list of particles for each box. If the box size is chosen wisely it is possible to find all neighboring particles by just looking in the adjacent boxes, which means that the loop over all j for each i may be avoided.

6.3 Velocity correlation

Whereas static quantities—the ones that may be determined from configuration snapshots—should be the same independent of α , dynamic quantities should be different for different α . The task is now to determine the velocity auto-correlation function $g_v(t)$

$$g_v(t) = \langle \mathbf{v}_i(t') \cdot \mathbf{v}_i(t' + t) \rangle, \quad (8)$$

Determine $g_v(0.0)$, $g_v(0.1)$, $g_v(0.2)$, $\dots$, $g_v(5.0)$ at $\rho = 0.2$ (a lower density than before), $T = 1.0$, and $\alpha = 0.01, 0.1$, and 1.0 and show them in the same plot.

Hint: Use a circular buffer to store the velocity vector for 50 earlier times and create a file to write out the velocity correlations. It could be convenient to put the functions for initialisation, accumulation, and for writing out results related to the velocity correlation in a file `vcorr.c` together with a header file `vcorr.h`, which should be included from both `sim.c` and `vcorr.c`

A Program overview and some hints for the coding

A.1 Data storage

There are several different ways to store the positions and velocities for a set of particles, but we here go for the simplest which is just to use arrays of type `double`. One reason for this choice is that it is then easy to write code with or without velocity coordinates which can be used in both two and three dimensions.

With the variable `pos` for the positions, the x and y , coordinates of the first particle, particle number zero, are then `pos[0]` and `pos[1]`, and it follows that the storage for the second particle start at `pos[2]`, or, more generally, with D for the dimensionality, storage for particle i starts at `pos[D * i]`.

Since some of our simulation methods uses velocity variables whereas others do not, we want the velocity variable, `vel`, to be present only in certain versions of the program. We then let the first half of the array `atoms` contain the position array whereas the second part contains the velocities. With the number of particles in `par->n`, a part of the code may then be written

```
double *atoms;
atoms = malloc(par->n * 2 * D * sizeof(double));

double *pos = atoms;
double *vel = atoms + par->n * D;
```

The notation in “`double *pos = atoms`” is confusing, but the declaration should be taken to mean that `pos` is of type `double *`, a pointer to a double. This is thus actually the same as

```
double *pos;
pos = atoms;
```

To illustrate the use of these vectors we show the code to advance the position of the particles one time step. We can then either write

```
for (i = 0; i < par->n; i++) {
```

```

    for (d = 0; d < D; d++)
        pos[D * i + d] += par->deltat * vel[D * i + d];
}

```

or, by just combining in a single loop

```

for (id = 0; id < D * par->n; id++)
    pos[id] += par->deltat * vel[id];

```

or, by using `ipos` and `ivel` for position and velocity of particle `i`,

```

for (i = 0; i < par->n; i++) {
    double *ipos = pos + D * i;
    double *ivel = vel + D * i;
    for (d = 0; d < D; d++)
        ipos[d] += par->deltat * ivel[d];
}

```

Note that the code above is a not complete, as the particles will now and then move outside the simulation box. One therefore needs to include a check that the new position is within the range $[0, L)$, and otherwise move it to the image position that is inside the simulation box. This test should be done each time a particle is moved.

A.2 Code overview

Sketch of the contents of `sim.c`:

```
void run_simulation(Par *par)
{
    .           // Initialize
    .
    for (iblock = 0; iblock < par->nblock; iblock++) {
        for (isamp = 0; isamp < par->nsamp; isamp++) {
            for (istep = 0; istep < nstep; istep++) // Advance one time unit
                step(par, atoms, force);           // Take one leap-frog step

            measure(par, atoms, val);               // Measure quantities, store in vblock
        }
    }
    .
}

// read_args interprets the commands on the command line
int read_args(Par *par, char *arg)
{
    .
    if (!strcmp(arg, "run")) {
        .
        run_simulation(par);
    }
    .
}

// The main program. Execution starts here.
int main(int argc, char *argv[])
{
    for (iarg = 1; iarg < argc; iarg++)
        if (!read_args(&par, argv[iarg]))
            exit(EXIT_FAILURE);
    .
    exit(EXIT_SUCCESS);
}
```

B Running several simultaneous jobs

Below are some hints for speeding things up. You don't need to follow them.

With 8 different densities and 6 temperatures these runs could be rather cumbersome. Since modern computers typically have four cores it is however possible to speed things up by running several jobs at the same time. (This is even better done on the server, `sesam`, which has 12 cores. From the computer lab, log in with “`ssh sesam`”.) To run several jobs at the same time one can make use of the built-in loop feature in the `bash` shell. As a simple demonstration try

```
for r in 0.15 0.20 0.25; do echo $r; done
```

In our case this can be combined with the line above to give (but change “...” and keep it on a single line)

```
for r in 0.15 0.20 ... 0.80; do ./sim N=64 rho=$r  
T=1.000 read T=0.900 nblock=100 run & done
```

Note that the `read` command will fail if there is no file in the `conf` directory for the specified parameters.

The construct “`./sim...run & done`” starts the runs in parallel. If one had instead written “`./sim...run ; done`” the runs would be started sequentially, one after the other, which is not what we want.

A drawback with this approach is that it mixes the output from the programs. To avoid that one can redirect the output from each run to a separate file:

```
for r in 0.15 0.20 ... 0.80; do ./sim N=64 rho=$r T=1.000  
read T=0.900 nblock=100 run > log/0064-r${r}_T0.900 & done
```

Don't forget to first create the `log` directory.